

Reflect, Revise, Reuse: Training-Free Skill Evolution for GUI Agents

Bofan Chen^{1*} Boxuan Zhang^{1,2*} Fei Tang¹ Zhengxi Lu¹ Yong Du¹
 Tongbo Chen¹ Weiming Lu¹ Jun Xiao¹ Yueting Zhuang¹ Yongliang Shen^{1†}

¹Zhejiang University ²UESTC

{sy1}@zju.edu.cn

 Code  Project Page

Abstract

GUI agents execute long-horizon tasks on dynamic graphical user interfaces, where pop-ups, delayed loads, and relocated widgets routinely invalidate plans fixed before execution. Recent agent-skill frameworks encapsulate reusable procedural knowledge to mitigate this, yet existing skill designs are largely developed without targeting GUI execution dynamics and treat skills as static artifacts produced before deployment rather than living procedural knowledge that improves through it. We argue that what GUI agents need is not better static skills, but skills that can be revised from execution feedback at deployment time, without additional training. We propose **EvoSkill-GUI**, a training-free framework in which each skill is a structured multi-file package containing retrieval metadata, executable plans, backup localization, failure-recovery rules, accessibility utilities, and failure cases. EvoSkill-GUI operates through a *reflect-revise-reuse* loop: the executor performs instant in-rollout revisions, an isolated critic diagnoses failed trajectories under strict information isolation, and the executor edits specific skill files through a restricted tool interface. Across MobileWorld, AndroidWorld, and OSWorld, three mainstream GUI benchmarks spanning mobile and desktop platforms, EvoSkill-GUI consistently improves multiple base models without any training, with maximum gains of +16.2%, +6.0%, and +10.5% respectively, and evolved skill libraries continue to benefit related tasks rather than being rebuilt from scratch. Our code is available at <https://github.com/ZJU-REAL/EvoSkill-GUI>.

digital workflows on mobile and desktop applications. Unlike single-step execution tasks (Cheng et al., 2024; Li et al., 2025a; Lu et al., 2025; Tang et al., 2025a), real GUI execution unfolds in non-stationary environments where pop-ups, delayed loads, and relocated widgets routinely invalidate plans fixed before execution (Li et al., 2025b; Zhang et al., 2026c). Agent-skill frameworks (Anthropic, 2025; Zhang et al., 2026a; Xia et al., 2026; Xu and Yan, 2026) provide a natural mechanism for reusing procedural knowledge across related tasks, yet existing skill designs are largely developed without targeting GUI execution dynamics, and treat skills as static artifacts produced before deployment rather than living procedural knowledge that improves through it.

Executing long-horizon GUI tasks under current skill-based paradigms encounters four barriers that existing designs (Zhang et al., 2026b; Chen et al., 2026; Wang et al., 2026) do not jointly address. **First, skills are often unstructured and hard to edit.** When procedural knowledge is stored as a single long document, planning steps, localization hints, and recovery rules are entangled, making targeted revision difficult. **Second, interfaces are non-stationary.** A skill valid at one interface state may fail when a pop-up appears, a widget shifts, or an accessibility tree becomes stale, and idempotent failure loops account for a substantial portion of timeouts in real GUI deployments (Zhang et al., 2026c; Wu et al., 2025). **Third, external reflection introduces extra cost and weak coupling.** Relying on an external reflection model adds latency and leaves reflection loosely coupled with skill execution and revision, making the system less fully self-evolving. **Fourth, procedural knowledge does not accumulate.** Lessons about unreliable selectors, missing contingencies, and required backtracks remain locked inside individual trajectories rather than becoming reusable assets for nearby tasks (Fang et al., 2026; Allard et al.,

1 Introduction

GUI agents (Tang et al., 2025b; Nguyen et al., 2025; Ye et al., 2025a) promise to automate long-horizon

* Equal contribution.

† Corresponding author.

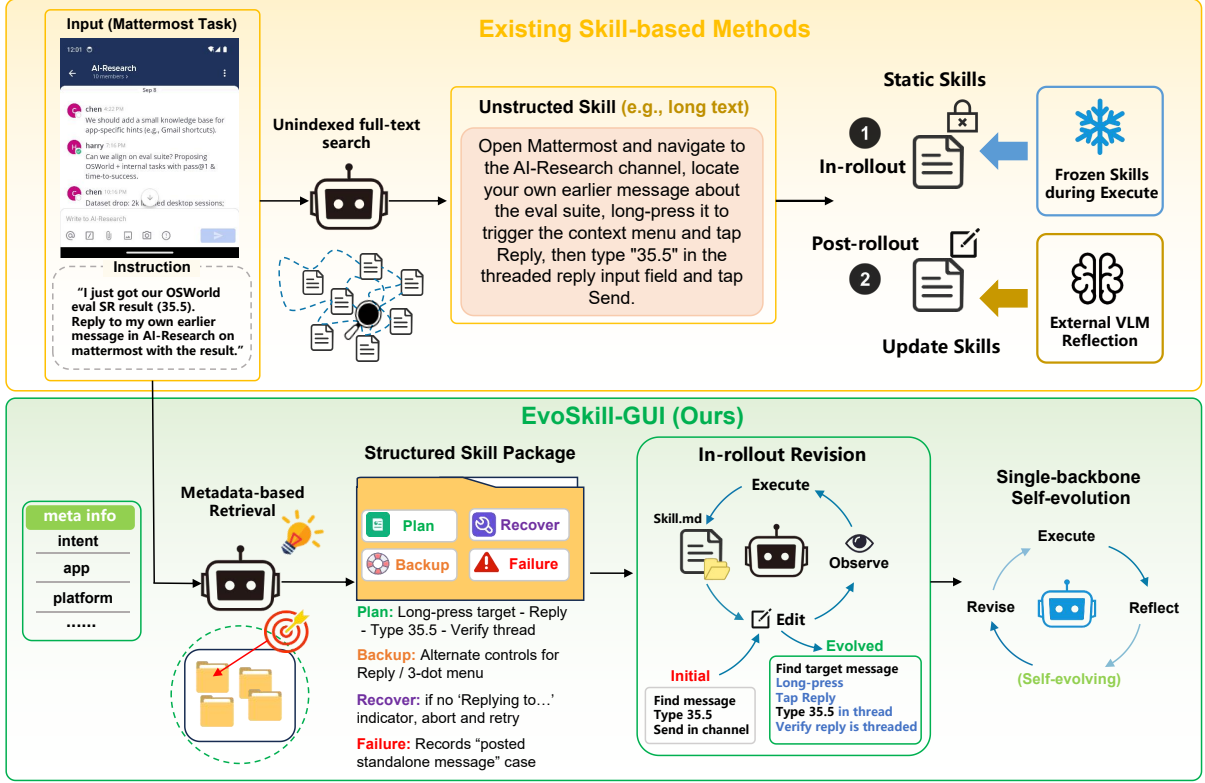


Figure 1: Comparison between existing skill-based methods and **EvoSkill-GUI**. The upper panel illustrates the limitations of existing approaches, which rely on unstructured single-file skills, static skills during execution, external-VLM reflection, and unindexed full-text retrieval. The lower panel shows how **EvoSkill-GUI** addresses these issues with structured editable skill packages, metadata-based retrieval, in-rollout revision, and single-backbone self-evolution, enabling skills to be adapted and reused from real execution feedback.

2026). Together, these limitations indicate that GUI agents need skills that can not only be reused, but also revised from execution feedback.

Many GUI-agent failures carry actionable information about which part of a skill is wrong: grounding errors expose unreliable localization, unexpected pop-ups expose missing recovery branches, and redundant steps expose flawed plans. Instead of discarding such failures or using them only for future training (Sun et al., 2025; Xue et al., 2026), we ask **whether a deployed agent can immediately reflect on the failure and write the resulting procedural correction back into the skill itself**. Unlike concurrent self-evolving skill frameworks targeting code-centric environments (Zhang et al., 2026a) or structured agent memory updated through retrieval rather than skill-level revision (Zhu et al., 2026; Fang et al., 2026; Luo et al., 2026), this yields a training-free formulation in which skills become structured, editable packages that accumulate task-specific experience over repeated use.

To realize this idea, we propose **EvoSkill-GUI**, a training-free self-evolving framework built around

a **reflect-revise-reuse** loop. Each skill is represented as a structured multi-file package containing retrieval metadata, executable plans, backup localization strategies, failure recovery rules, accessibility utilities, and failure examples. During rollout, the agent performs instant revisions when feedback contradicts the current plan. After a failed rollout, the same backbone model performs **single-backbone self-evolution**: it acts as an isolated critic to diagnose the trajectory under strict information isolation, and then as an executor to edit specific files of the package through a restricted tool interface so that revisions remain localized and auditable. Verified packages enter a library indexed by structured metadata, letting related tasks start from procedural knowledge that has already been validated.

We evaluate **EvoSkill-GUI** on MobileWorld, AndroidWorld, and OSWorld, three GUI benchmarks spanning mobile and desktop environments. Across both general-purpose and GUI-specialized base models, **EvoSkill-GUI** consistently improves task success rates without any training, with maximum gains of +16.2%, +6.0%, and +10.5% re-

spectively. Ablations confirm the importance of structured packages, instant revision, and reflection-driven edits, and analyses indicate that evolved skill libraries continue to benefit related tasks rather than being rebuilt from scratch.

Our contributions are threefold:

- We introduce EvoSkill-GUI, a training-free framework that enables GUI agents to construct, retrieve, execute, and revise self-evolving skill packages through information-isolated reflection and tool-restricted edits.
- We design a **GUI-oriented structured multi-file skill package** that separates retrieval metadata, executable plans, backup localization, recovery rules, accessibility utilities, and failure examples, enabling skills to be reused and updated at inference time.
- We demonstrate consistent improvements on MobileWorld, AndroidWorld, and OSWorld across multiple base models, with gains up to +16.2%, +6.0%, and +10.5% respectively.

2 Related Work

2.1 Agent Skills

Agent skills (Anthropic, 2025; Zhou et al., 2026; Xu and Yan, 2026) externalize reusable procedural knowledge beyond model parameters, representing skills as structured artifacts with workflows, parameterized execution, and composition rules. CUA-Skill (Chen et al., 2026) builds a structured desktop skill base, WebXSkill (Wang et al., 2026) couples executable web programs with step-level guidance, and MMSkills (Zhang et al., 2026b) extends skills with multimodal visual evidence. XSkill (Jiang et al., 2026) combines experiences and skills into a dual-stream continual learning framework updatable from past trajectories in a training-free manner. Structured memory systems such as HyMem (Zhao et al., 2026) and its GUI extension (Zhu et al., 2026) organize reusable experience but focus on memory construction rather than skill-level revision. At the learning level, SkillRL (Xia et al., 2026), D2Skill (Tu et al., 2026), Skill0 (Lu et al., 2026b), and SDAR (Lu et al., 2026a) explore skill distillation and internalization through reinforcement learning. The concurrent CoEvoSkills (Zhang et al., 2026a) co-evolves multi-file skill packages with a surrogate verifier, but targets code-centric environments and skill construction rather than deployed

GUI skill revision. Overall, existing systems either construct skills offline, evolve them only during training, or keep them static after retrieval, leaving inference-time skill revision largely unexplored.

2.2 Self-evolving Agents

A separate line studies how agents improve from their own interaction experience. In GUI settings, UI-Evol (Zhang et al., 2025) refines external knowledge by retracing trajectories against reference knowledge, UI-Mem (Xiao et al., 2026) maintains hierarchical experience memory over online reinforcement learning, and Mobile-Agent-E (Wang et al., 2025b) and SEAgent (Sun et al., 2025) evolve via persistent memory and reflective modules. EvoCUA (Xue et al., 2026) couples synthetic experience generation with iterative policy evolution, turning failures into supervision through error analysis. More recently, UI-Voyager (Lin et al., 2026) learns from failed mobile-GUI trajectories via group-relative self-distillation, MobileUse (Li et al., 2025b) adds hierarchical reflection at step and trajectory level, and GUI-Reflection (Wu et al., 2025) embeds reflection supervision across pre-training, SFT, and online tuning. Beyond GUI, Memp (Fang et al., 2026) and Experiential Reflective Learning (Allard et al., 2026) maintain procedural memory or reusable heuristics from past trajectories, alongside web and general-agent self-improvement (Fang et al., 2025; Lin et al., 2026; Zhao et al., 2026; Luo et al., 2026). These methods show that experience feedback supports continual improvement, but typically update memory, policy, or model weights rather than the deployed skill artifact itself. In contrast, EvoSkill-GUI revises structured GUI skill packages from failure feedback, enabling self-evolution at the skill level without additional training.

3 Method

3.1 Problem Formulation

Following a prior skill-evolution work’s formulation (Zhang et al., 2026a), we model GUI skill evolution as the problem of optimizing a reusable skill package under partial observability. A GUI task is modeled as a partially observable Markov decision process $\mathcal{M} = \langle \mathcal{X}, \mathcal{A}, T, \mathcal{O}, \Omega, R \rangle$, where $\mathcal{X}$ denotes the underlying screen states, $\mathcal{A}$ denotes GUI actions and tool calls, T is the transition function, and $R(x_T) \in \{0, 1\}$ is the terminal task-completion reward. At each step, the agent ob-

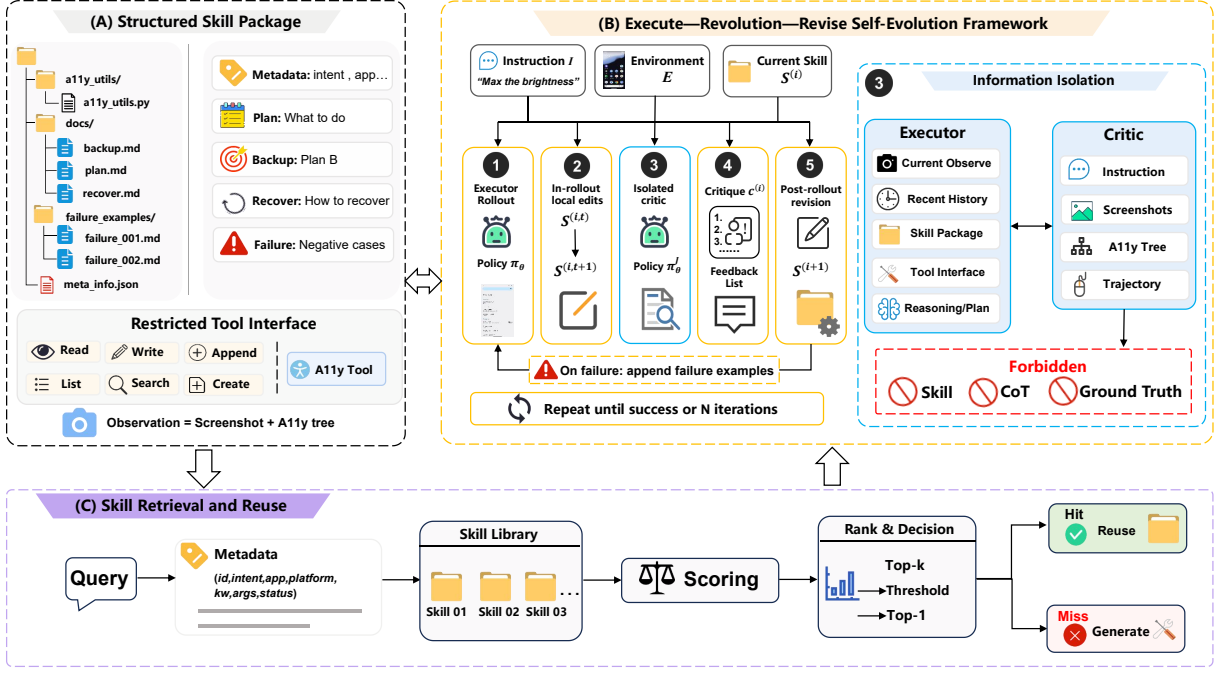


Figure 2: Overview of **EvoSkill-GUI**. (A) Each skill is a structured multi-file package containing accessibility utilities, planning, backup localization, recovery rules, failure cases, and metadata, accessed through a restricted tool interface. (B) The execute—reflect—revise loop alternates rollouts with in-rollout edits, isolated post-rollout critique, and skill-file revisions, with strict information isolation between executor and critic. (C) Evolved skills are indexed by metadata and reused via threshold-based scoring; misses trigger new skill generation.

serves $o_t = (\varphi_t, \zeta_t)$ consisting of a screenshot φ_t and an accessibility tree ζ_t , and acts based on the partial history $h_t = (o_1, a_1, \dots, a_{t-1}, o_t)$.

A skill package S is a persistent procedural knowledge object that conditions the agent policy:

$$a_t \sim \pi_\theta(a_t \mid h_t, S). \quad (1)$$

Given an instruction I , the expected task return of S is

$$J(S, I) = \mathbb{E}_{\tau \sim P(\tau \mid \pi_\theta, S, \mathcal{M}, I)} [R(x_T)], \quad (2)$$

where τ is the execution trajectory. For a task distribution $\mathcal{D}$, EvoSkill-GUI aims to obtain a reusable skill package that improves expected return across related tasks:

$$S^* = \arg \max_S \mathbb{E}_{I \sim \mathcal{D}} [J(S, I)]. \quad (3)$$

The remaining question is how to represent S and how to revise it without training or external supervision.

3.2 Skill Package Representation

EvoSkill-GUI represents each skill as a structured and editable package rather than a single free-form prompt. Formally, a skill package is

$$S = (D, A, P, B, C, F), \quad (4)$$

where D is retrieval metadata, A is accessibility tree related utilities, P stores executable planning knowledge, B stores backup localization and recognition strategies, C stores failure-recovery rules, and $F = \{f^{(1)}, \dots, f^{(n)}\}$ stores failure cases. This decomposition aligns the package with three common sources of GUI-agent error: P specifies what steps to perform, B specifies how to identify and locate interface elements, and C specifies how to recover when the expected state is violated. Compared with a monolithic skill file, this structure makes revisions more targeted: a grounding failure can update B , a missing contingency can update C , and a high-level procedural error can update P .

Tool interface. To make the package editable during execution, EvoSkill-GUI exposes the skill directory through a restricted tool interface

$$\mathcal{T} = \{\text{read, write, append, list, search, create_failure}\}. \quad (5)$$

These tools allow the agent to inspect and revise the predefined skill files while preventing arbitrary modifications outside the package schema. In addition, a read-only accessibility tool ψ returns the accessibility tree of the current window. Since ψ only augments observation and does not modify

S , EvoSkill-GUI separates interface understanding from skill revision at the tool level.

3.3 Reflect-Revise Loop

The loop alternates three phases: rollout with instant in-rollout revisions, isolated post-rollout reflection, and skill-file revision based on the critique.

Rollout and instant revision. At iteration i , the executor runs the current skill package $S^{(i)}$ in the environment and obtains a trajectory

$$\tau^{(i)} = \Phi(S^{(i)}, \mathcal{E}), \quad (6)$$

where Φ denotes rollout generation until success, failure, or the step limit. During the rollout, the executor may invoke tools in $\mathcal{T}$ to perform instant revisions, producing an intermediate package $S^{(i, \text{end})}$. This allows the agent to correct local mismatches such as relocated widgets or unexpected pop-ups before they cascade into full task failure.

Reflection. After each rollout, EvoSkill-GUI uses the same backbone model as an isolated critic π_θ^J to diagnose the trajectory:

$$c^{(i)} \sim \pi_\theta^J(\cdot \mid I, o_{1:T}^{(i)}, a_{1:T}^{(i)}). \quad (7)$$

The critic outputs structured feedback, including failure-step localization, direct-cause analysis, and actionable revision suggestions. To prevent the critic from relying on privileged information unavailable to a deployed agent, we require the critic’s information set $\mathcal{C}_J$ to be strictly contained in the executed trajectory:

$$\mathcal{C}_J \subseteq \{I\} \cup o_{1:T} \cup a_{1:T}, \quad (8a)$$

$$\mathcal{C}_J \cap \{S, \text{CoT}_\theta, \text{GT}\} = \emptyset, \quad (8b)$$

$$\mathcal{C}_E \supseteq \mathcal{C}_J \cup \{S, \text{CoT}_\theta\}. \quad (8c)$$

The critic shares parameters with the executor but runs in a separate session, so improvements cannot be attributed to a stronger external supervisor.

Revision. Given the rollout, the post-rollout skill snapshot, and the critic feedback, the executor revises the skill package:

$$S^{(i+1)} \sim \pi_\theta(\cdot \mid S^{(i, \text{end})}, c^{(i)}, \tau^{(i)}). \quad (9)$$

If the rollout fails, EvoSkill-GUI appends a failure case $f^{(i)} = (I, c^{(i)}, \tau^{(i)})$ to F through `create_failure`. The loop terminates when the task succeeds or when the maximum number of revision rounds is reached. In-rollout edits thus handle local execution mismatches, while post-rollout critique handles structural errors requiring trajectory-level diagnosis.

3.4 Skill Retrieval and Reuse

Evolved skills show much more value if they can be retrieved for later related tasks. EvoSkill-GUI therefore indexes each skill by structured metadata rather than by its full procedural body:

$$D = (\text{id}, \text{intent}, \text{app}, \text{platform}, \text{kw}, \text{args}, \text{hist}, \text{status}). \quad (10)$$

where intent describes the skill goal, app and platform specify the execution context, kw stores retrieval keywords, args records reusable input slots, hist stores usage history, and status indicates whether the skill is verified. Indexing by metadata avoids matching against long procedural files whose surface tokens may overlap even across distinct tasks.

Retrieval score. For a new instruction q , EvoSkill-GUI scores each candidate skill by combining lexical coverage, semantic tightness, and lightweight task constraints. With query tokens Q , skill tokens T_S , and overlap $O = Q \cap T_S$, the base score uses empirically chosen weights:

$$\text{base}(q, S) = 0.6 \frac{|O|}{|Q|} + 0.4 \frac{|O|}{|Q \cup T_S|}. \quad (11)$$

The final score adds bonuses for matched app and keyword fields and subtracts a divergence penalty for query-specific concept tokens that are missing from the candidate:

$$\text{score}(q, S) = \text{clip}(\text{base} + b_{\text{app}} + b_{\text{kw}} - p_{\text{div}}, 0, 1). \quad (12)$$

Reuse. At inference time, EvoSkill-GUI ranks the skill library by Eq. (12). If the top candidate exceeds the retrieval threshold θ_r , the agent reuses the corresponding skill package; otherwise, it creates a new package from scratch. A newly created package enters the library only after successful execution or after being revised through the self-evolution loop. Thus, future tasks can start from verified procedural knowledge instead of repeatedly rebuilding similar skills from zero.

4 Experiments

4.1 Experimental Setup

Benchmarks We evaluate EvoSkill-GUI on three interactive GUI benchmarks covering both mobile and desktop environments. AndroidWorld (Rawles et al., 2025) provides a fully functional Android environment with 116 tasks across 20 real-world

apps, enabling reproducible evaluation under dynamically parameterized instructions. MobileWorld (Kong et al., 2025) is a more challenging mobile-use benchmark designed to reflect realistic app usage through long-horizon, cross-app workflows. Since our focus is GUI interaction, we evaluate MobileWorld on GUI-only setting. OSWorld (Xie et al., 2024) provides a scalable real computer environment spanning Ubuntu, Windows, and macOS, supporting execution-based evaluation across arbitrary applications. Together, these benchmarks test whether EvoSkill-GUI can improve agents across platforms, applications, and task types.

Models We evaluate our method using two categories of base models: (1) general-purpose closed-weight models and open-weight models, including Claude-Sonnet-4.6 (Anthropic, 2026), Qwen3.6-Plus, and Qwen3.6-35B-A3B (Qwen Team, 2026), which provide strong general multimodal reasoning; and (2) GUI-specialized open-weight models, including GUI-Owl-1.5-8B (Ye et al., 2025b), and MAI-UI-8B (Zhou et al., 2025), which are tailored for GUI understanding and grounding. This setup allows us to examine whether EvoSkill-GUI can consistently benefit both strong general agents and models already adapted for GUI interaction.

Implementations For closed-weight models and large-parameter open-weight models, we use the official APIs provided by their corresponding providers. For smaller open-weight models, we deploy them on one RTX PRO 6000 GPU 96GB using vLLM; the detailed deployment configuration is provided in the Appendix A.1. Across all models and benchmarks, we keep the skill package format and retrieval strategy fixed. Each task is allowed up to 50 interaction steps, and after a failed attempt, the agent is allowed at most two rounds of skill revision. The prompts used in our experiments are also provided in the Appendix A.8.

4.2 Main Results

EvoSkill-GUI improves multiple base models on mobile GUI tasks. On the GUI-only MobileWorld subset (Figure 3), EvoSkill-GUI improves every evaluated base model. For closed-weight models, it raises Claude-Sonnet-4.6 from 57.1% to 67.6% and Qwen3.6-Plus from 53.3% to 69.5%. For open-weight models, it raises Qwen3.6-35B-A3B from 32.4% to 44.8% and MAI-UI-8B from 29.5% to 37.1%. These consistent gains across

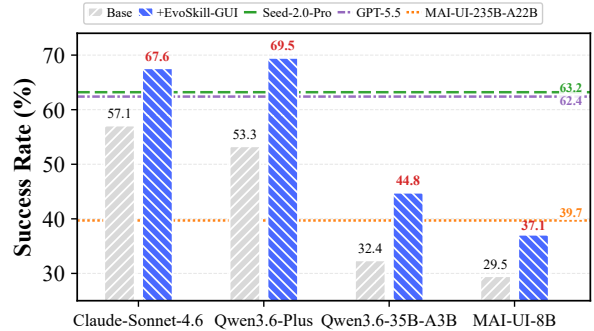


Figure 3: Performance comparison on MobileWorld under the GUI-only setting. The horizontal lines indicate representative reference baselines.

model families and scales indicate that structured skill packages provide procedural guidance robust to the specific backbone.

Beyond MobileWorld, EvoSkill-GUI also improves AndroidWorld performance (Table 2), raising success rates by +2.6% under seed 30 and +6.0% under seed 42.

EvoSkill-GUI extends to desktop computer-use tasks. Table 1 reports OSWorld performance on two base models. On GUI-Owl-1.5-8B, EvoSkill-GUI raises the overall success rate from 46.7% to 54.8% (+8.1), with large gains on applications such as Thunderbird (+20.0), VLC (+42.7), and VS Code (+5.2). On Qwen3-VL-8B-Instruct, the overall score improves from 23.8% to 34.3% (+10.5). These results show that the skill package design transfers beyond mobile environments to heterogeneous desktop workflows. We discuss the two negative domain-level entries and their likely causes in Appendix A.4.

4.3 Ablation Study

We conduct systematic ablation experiments on MobileWorld to validate the design of each component in EvoSkill-GUI.

Structured skill packages outperform monolithic skill files. Table 3 compares the structured multi-file package with a single-file skill representation. The structured package improves success rate from 66.67% to 69.52% (+2.85). This supports our decomposition design: a monolithic file conflates planning, localization, and recovery, making targeted revision difficult, while the structured package allows each failure to update only the relevant component.

Instant in-rollout revision prevents cascading failures. Table 4 shows that removing instant re-

Method	Chrome	GIMP	LibreOffice			Multi-Apps	OS	Thunderbird	VLC	VS Code	Overall
			Calc	Impress	Writer						
General Closed-Weights Models											
Claude-Sonnet-4.5 (Anthropic, 2026)	56.4	57.7	66.0	57.5	65.2	47.0	70.8	66.7	52.9	69.6	58.1
Qwen3-VL-Flash (Bai et al., 2025)	56.4	42.3	23.0	51.0	56.5	22.0	54.2	66.7	34.4	69.6	41.6
Specialized Open-Weights Models											
UI-TARS-72B-DPO (Qin et al., 2025)	33.2	61.5	12.8	25.4	43.5	6.7	33.3	33.3	23.5	47.8	25.9
Computer-Use-Preview	36.9	34.6	14.9	29.7	26.1	15.8	70.8	66.7	11.8	69.6	31.2
OpenCUA-32B (Wang et al., 2025a)	45.6	65.4	12.8	42.5	39.1	14.6	60.9	46.7	23.5	65.2	35.1
EvoCUA-8B (Xue et al., 2026)	54.3	84.6	29.8	38.2	52.2	25.5	75.0	73.3	44.8	65.2	46.1
EvoCUA-32B	65.1	76.9	55.3	59.5	69.6	27.9	78.3	80.0	49.4	87.0	56.7
GUI-Owl-1.5-32B (Ye et al., 2025b)	66.6	76.9	48.9	63.8	60.9	27.3	76.2	86.7	37.4	87.0	55.4
Skill-based Works											
MMSkills (Zhang et al., 2026b)	39.9	42.3	8.5	23.4	17.4	13.4	25.0	29.4		47.8	25.4
Our Method											
GUI-Owl-1.5-8B	35.6	80.8	29.8	57.5	69.6	21.5	70.8	73.3	43.8	81.8	46.7
+EvoSkill-GUI	41.3	88.5	37.8	65.8	69.6	30.5	66.7	93.3	86.5	87.0	54.8
ΔImprovement	+5.7	+7.7	+8.0	+8.3	+0.0	+9.0	-4.1	+20.0	+42.7	+5.2	+8.1
Qwen3-VL-8B-Instruct	28.1	26.9	19.1	31.8	47.8	7.7	16.7	60.0	35.3	26.1	23.8
+EvoSkill-GUI	43.4	46.2	29.8	34.4	52.2	15.2	41.7	46.7	50.6	47.8	34.3
ΔImprovement	+15.3	+19.3	+10.7	+2.6	+4.4	+7.5	+25.0	-13.3	+15.3	+21.7	+10.5

Table 1: Performance comparison on the OSWorld-Verified benchmark across two models: GUI-Owl-1.5-8b and Qwen3-VL-8B-Instruct. For a fair comparison, all results are evaluated under a maximum of 50 steps.

Phase	Reuse Rate	Base SR	Ours SR	Δ
P1 (seed 30)	37.9	68.1	70.7	<i>+2.6</i>
P2 (seed 42)	100.0	55.2	61.2	<i>+6.0</i>

Table 2: Evaluation on the AndroidWorld task set. SR denotes success rate.

Skill Organization	Success Rate (%)
Single-file Skill	66.67
Skill Package (Ours)	69.52
<i>Δ Improvement</i>	<i>+2.85</i>

Table 3: Ablation of skill organization on MobileWorld.

vision decreases MobileWorld success rate from 69.52% to 62.86% (−6.66). Relying on a fixed pre-generated plan throughout a long-horizon GUI task is unreliable, because intermediate states frequently diverge from the agent’s original expectation by pop-ups, delayed loading, or changed widget positions. Instant revision mitigates such mismatches before they cascade into full task failure.

Metadata-based retrieval outperforms full-text matching. On twelve MobileWorld reuse cases shown in Table 5, all completed successfully after retrieving the correct skill, metadata-based retrieval achieves an average score of 0.88 compared with 0.41 for full-text retrieval. Under retrieval threshold $\theta_r = 0.6$, metadata-based retrieval recovers the correct skill in all 12 cases, while full-text retrieval succeeds in only one. This supports structured metadata thus provides a strong retrieval signal that

Strategy	Success Rate (%)
EvoSkill-GUI (Ours)	69.52
w/o information isolation	60.95
w/o instant revision	62.86

Table 4: Ablation of information isolation and instant revision on MobileWorld on Qwen3.6-Plus.

long full-text files, with overlapping surface tokens across distinct tasks, cannot reliably support.

Information isolation improves reflection quality. Table 4 shows that removing information isolation drops MobileWorld success rate from 69.52% to 60.95% (−8.57). By preventing the critic from relying on privileged context or inheriting the executor’s mistaken plan, information isolation produces more evidence-grounded diagnoses and leads to more reliable skill revision.

Each skill-package component contributes to performance. Figure 5a ablates skill-package components. Removing planning documents reduces success rate from 69.52% to 65.71% (−3.81), the largest drop, indicating that explicit procedural plans are critical for maintaining a coherent action sequence across long horizons. Removing failure reflection also drops performance to 66.67% (−2.85), showing that post-failure diagnosis provides useful corrective knowledge for subsequent executions.

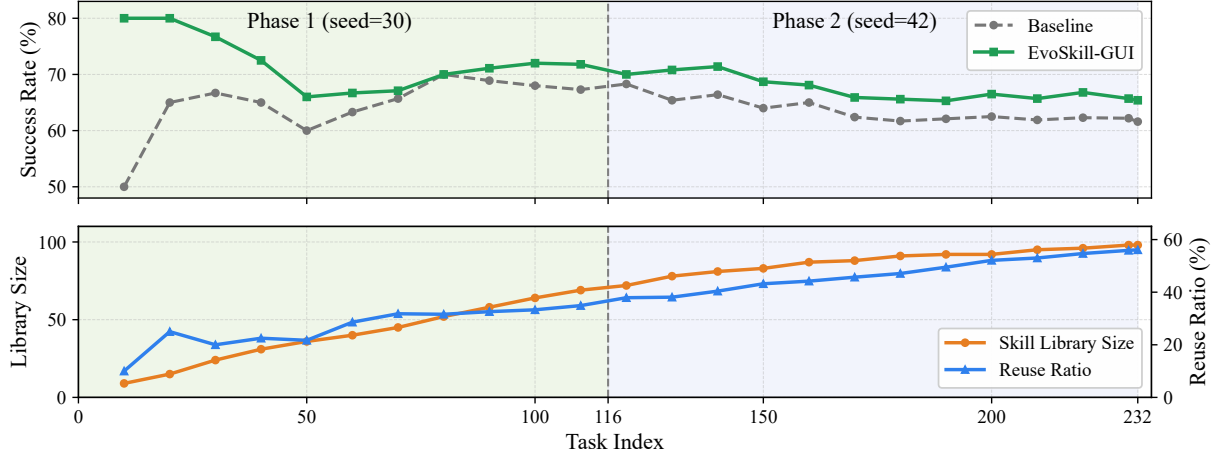


Figure 4: Skill-library growth on the 116+116 AndroidWorld task set. EvoSkill-GUI maintains higher cumulative success rates than the baseline as the library grows and the reuse ratio increases.

Task name	Metadata Score	Full-text Score	Meta@0.6	Full-text@0.6
Mastodon New Post	0.8036	0.3431	✓	✗
Mattermost Create Channel	1.0000	0.4769	✓	✗
TaoDian Item Checkout	1.0000	0.6182	✓	✓
Mastodon Reply (Moussaka)	1.0000	0.3833	✓	✗
Change Wallpaper (Sunflower)	0.8333	0.4184	✓	✗
Mattermost Send File	0.7003	0.3414	✓	✗
Mattermost Reply Result	0.7833	0.3521	✓	✗
Check Cart Price	0.9500	0.4231	✓	✗
Mastodon Unfollow	0.8167	0.3876	✓	✗
Mastodon Save Photos	0.8500	0.3654	✓	✗
Google Maps Alibaba South	0.9167	0.4512	✓	✗
Mastodon Conditional Favorite	0.8500	0.2987	✓	✗
Average / Hit Rate	0.8753	0.4050	12/12	1/12

Table 5: Ablation of metadata for skill retrieval on MobileWorld reuse cases, all cases are completed successfully by EvoSkill-GUI. @0.6 indicate whether the corresponding retrieval score exceeds the threshold $\theta_r = 0.6$.

Accessibility-tree utilities benefit grounding across models. Figure 5b shows that adding accessibility-tree inputs improves Qwen3.6-Plus by +3.80, Qwen3.6-35B-A3B by +1.90, and MAI-UI-8B by +2.85. The consistent gains across backbone families suggest that accessibility-tree utilities act as a structured observation channel complementary to vision, supplying actionable grounding cues regardless of the model’s GUI pretraining.

A moderate retrieval threshold balances skill reuse and selectivity. Table 8 analyzes the sensitivity of EvoSkill-GUI to the retrieval threshold θ_r on MobileWorld. Lowering the threshold from 0.6 to 0.4 reduces the success rate from 69.5% to 55.2%, suggesting that overly permissive retrieval can introduce weakly related skill packages. Raising the threshold to 0.8 also decreases the success rate to 66.7%, indicating that overly conservative retrieval may miss useful reusable skills. The best

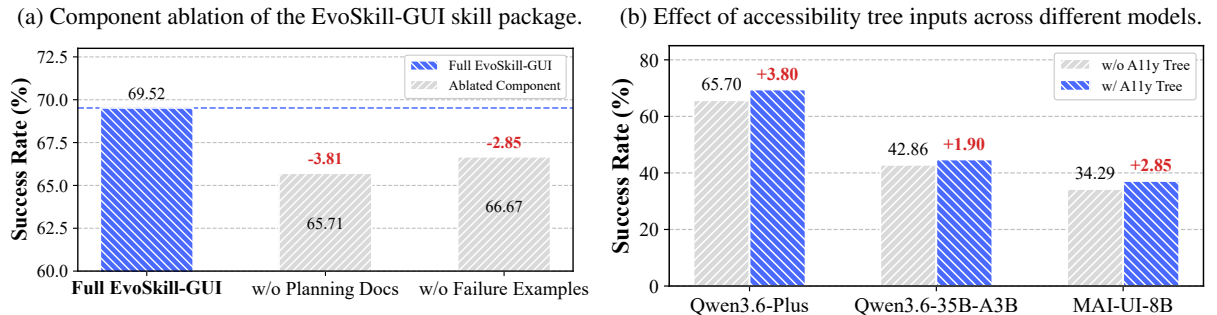


Figure 5: Ablation studies on skill-package components.

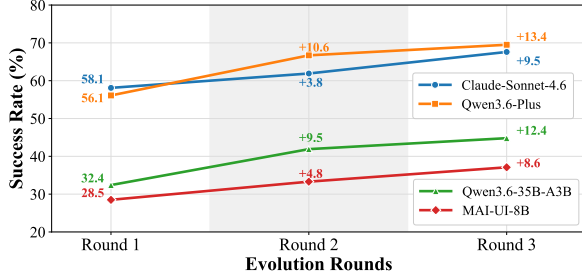


Figure 6: Three round evolution of multiple base models on MobileWorld.

performance at $\theta_r = 0.6$ supports its use as a balanced default in our experiments.

Retrieval Threshold θ_r	Success Rate (%)
0.4	55.2
0.6 (Ours)	69.5
0.8	66.7

Table 6: Sensitivity of MobileWorld performance to the retrieval threshold θ_r .

4.4 Analysis

Skill evolution provides cumulative gains and largely saturates after three rounds. Figure 6 shows that EvoSkill-GUI consistently improves success rates across rounds on MobileWorld. Claude-Sonnet-4.6 rises from 58.1% in Round 1 to 67.6% in Round 3. The open-weight models show the same trend: Qwen3.6-35B-A3B improves from 32.4% to 44.8%, and MAI-UI-8B from 28.5% to 37.1%. We further extend Qwen3.6-Plus to five rounds to examine saturation. Its success rate rises from 56.2% to 61.9% (+5.7) and 68.6% (+6.7) over the first three rounds, remains at 68.6% in Round 4, and reaches 69.5% in Round 5 (+0.9). Thus, the first three rounds yield a 12.4-point gain, while two additional rounds add only 0.9 point, supporting three rounds as an effective performance-cost trade-off.

Skill evolution outperforms repeated sampling under a comparable budget. Table 13 compares EvoSkill-GUI with a Qwen3.6-Plus pass@3 baseline on MobileWorld. While three independent baseline rollouts achieve 62.8% using 103.00M tokens, three-round EvoSkill-GUI achieves 69.5% using 94.75M tokens, improving accuracy by 6.7 points with a slightly lower budget. This comparison suggests that the gains arise from reflection-driven skill evolution rather than repeated sampling or increased token usage alone.

Skill Source	Tasks	Recovered	Recovery (%)
New skills	72	8/24	33.3
Reused skills	160	19/65	29.2
Overall	232	27/89	30.3

Table 7: Analysis of skill-reuse on failure-to-success recovery on the AndroidWorld.

Retrieval Threshold θ_r	Success Rate (%)
0.4	55.2
0.6 (Ours)	69.5
0.8	66.7

Table 8: Sensitivity of MobileWorld performance to the retrieval threshold θ_r .

Evolved skill libraries remain reusable and effective on related tasks. Tables 2 and 7 analyze whether EvoSkill-GUI accumulates reusable procedural knowledge across related AndroidWorld tasks. We build a 116+116 task set with seed-30 tasks as Phase 1 and related seed-42 variants as Phase 2. EvoSkill-GUI improves success rate from 68.1% to 70.7% in Phase 1, where 37.9% of tasks already reuse matched skills, and from 55.2% to 61.2% in Phase 2, where all tasks reuse the evolved library. Table 7 further shows that these reused skills remain effective: EvoSkill-GUI recovers 27 out of 89 initially failed executions overall, including 19 out of 65 failures by reused skills. These results indicate that the skill library accumulates reusable procedural experience over time, rather than rebuilding one-off task solutions from scratch.

Skill libraries accumulate reusable experience over task streams. Figure 4 shows EvoSkill-GUI on the sequential 116+116 AndroidWorld task set. In Phase 1, the library grows from 9 skills at task 10 to 69 skills at task 110, with reuse becoming more frequent as more matches become available. In Phase 2, the evolved library is applied to related seed-42 variants, further reaching 98 skills and a 56.1% reuse ratio. The higher cumulative success rates across both phases suggest that EvoSkill-GUI accumulates reusable procedural experience rather than isolated task solutions. This motivates the following recovery analysis.

Different failure types require different skill edits. We next ask which failures are actually repairable by skill revision. Table 9 shows that plan-level errors are the most repairable, with 16/23 recovered by revising plan.md; grounding errors are partially repairable through backup.md (6/12), while missing contingencies are harder to

Failure Type	Target File	Recovered	Rate
Plan-level error	plan.md	16/23	69.6
Grounding error	backup.md	6/12	50.0
Missing contingency	recover.md	1/4	25.0

Table 9: Repairability analysis of failed executions by target skill component.

fix through `recover.md` (1/4). Overall, this supports the structured design of **EvoSkill-GUI**: separating planning, grounding, and recovery knowledge lets revision target the component most responsible for a failure.

5 Conclusion

We presented **EvoSkill-GUI**, a training-free framework that turns deployed GUI skills from static artifacts into living procedural knowledge revisable from execution feedback. By representing each skill as a structured multi-file package and updating it by a reflect-revise-reuse loop with information-isolated critic and tool-restricted edits, **EvoSkill-GUI** enables a single backbone model to construct, retrieve, execute, and revise skills entirely at inference time. Across *MobileWorld*, *AndroidWorld*, and *OSWorld*, **EvoSkill-GUI** consistently improves multiple base models without training.

Limitations

EvoSkill-GUI relies on the backbone model to interpret screenshots, accessibility trees, and execution trajectories, and to produce structured revisions through tool calls. When the backbone’s perception or trajectory diagnosis is unreliable, the resulting skill edit may not capture the true cause of failure and can in principle introduce regressions into a previously verified package. Although the tool-restricted interface and information-isolated critique reduce this risk, our framework does not currently include a formal verifier that vetoes harmful skill edits. Our evaluation focuses on three representative GUI benchmarks, but real-world deployment involves more frequent interface updates, personalized configurations, network-dependent latencies, and privacy-sensitive states than current benchmarks reflect. The reuse rate observed on AndroidWorld is achieved on parameterized variants of the same task family, and transfer to genuinely unseen application categories may degrade more gracefully than this number suggests. Finally, the skill-evolution loop incurs additional inference cost per failed rollout, namely one critic call plus one revision call; this overhead is negligible compared with retraining or fine-tuning the backbone model, but in latency-critical deployment it should be amortized across reuse.

Ethics Statement

EvoSkill-GUI is designed for benchmark GUI task execution and does not collect personal user data in our experiments. However, GUI agents that operate real applications can affect user accounts, files, settings, or third-party services if deployed without proper safeguards, so practical deployment should include explicit user authorization, action confirmation for sensitive operations, sandboxed execution when feasible, and audit logs of agent behavior. Skill packages produced by EvoSkill-GUI also warrant review before reuse in high-stakes or privacy-sensitive environments, since an incorrectly revised skill may silently repeat an unintended action across many future invocations; skill-level versioning, human-in-the-loop verification at deployment, and explicit fail-safe rules are natural next steps for responsible adoption of self-evolving GUI agents. Finally, we used large language models solely to assist with writing polish and minor code suggestions during implementation; all research ideas, framework design, experimental protocols, analysis, and

conclusions, including the skill package design, the reflect-revise-reuse loop, the information-isolation protocol, and the retrieval mechanism, are the authors’ own.

References

- Marc-Antoine Allard, Arnaud Teinturier, Victor Xing, and Gautier Viaud. 2026. [Experiential reflective learning for self-improving llm agents](#). *Preprint*, arXiv:2603.24639.
- Anthropic. 2025. Agent skills overview. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>. Accessed: 2026-03-30.
- Anthropic. 2026. Claude sonnet 4.6 system card. <https://www.anthropic.com/claude-sonnet-4-6-system-card>.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhi-fang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, and 45 others. 2025. [Qwen3-vl technical report](#). *Preprint*, arXiv:2511.21631.
- Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, Suzhen Zheng, Hao Fan, Pashmina Cameron, Justin Wagle, and Kazuhito Koishida. 2026. [Cua-skill: Develop skills for computer using agent](#). *Preprint*, arXiv:2601.21123.
- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. [Seeclick: Harnessing gui grounding for advanced visual gui agents](#). *Preprint*, arXiv:2401.10935.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2026. [Memp: Exploring agent procedural memory](#). *Preprint*, arXiv:2508.06433.
- Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. 2025. [Webevolver: Enhancing web agent self-improvement with coevolving world model](#). *Preprint*, arXiv:2504.21024.
- Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. 2026. [Xskill: Continual learning from experience and skills in multimodal agents](#). *Preprint*, arXiv:2603.12056.
- Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, Zhidan Liu, Steven Hoi, and Yue Wang. 2025. [Mobileworld: Benchmarking autonomous mobile agents in agent-user interactive and mcp-augmented environments](#). *Preprint*, arXiv:2512.19432.

- Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. 2025a. [Screenspot-pro: Gui grounding for professional high-resolution computer use](#). *Preprint*, arXiv:2504.07981.
- Ning Li, Xiangmou Qu, Jiamu Zhou, Jun Wang, Muning Wen, Kounianhua Du, Xingyu Lou, Qiuying Peng, Jun Wang, and Weinan Zhang. 2025b. [Mobileuse: A gui agent with hierarchical reflection for autonomous mobile operation](#). *Preprint*, arXiv:2507.16853.
- Zichuan Lin, Feiyu Liu, Yijun Yang, Jiafei Lyu, Yiming Gao, Yicheng Liu, Zhicong Lu, Yangbin Yu, Mingyu Yang, Junyou Li, Deheng Ye, and Jie Jiang. 2026. [Ui-voyager: A self-evolving gui agent learning via failed experience](#). *Preprint*, arXiv:2603.24533.
- Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanqing Xiong, and Hongsheng Li. 2025. [Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning](#). *Preprint*, arXiv:2503.21620.
- Zhengxi Lu, Zhiyuan Yao, Zhuowen Han, Zi-Han Wang, Jinyang Wu, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026a. [Self-distilled agentic reinforcement learning](#). *Preprint*, arXiv:2605.15155.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026b. [Skill0: In-context agentic reinforcement learning for skill internalization](#). *Preprint*, arXiv:2604.02268.
- Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. 2026. [From storage to experience: A survey on the evolution of llm agent memory mechanisms](#). *Preprint*, arXiv:2605.06716.
- Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, Xintong Li, Jing Shi, Hongjie Chen, Viet Dac Lai, Zhouhang Xie, Sungchul Kim, Ruiyi Zhang, Tong Yu, Mehrab Tanjim, and 11 others. 2025. [Gui agents: A survey](#). *Preprint*, arXiv:2412.13501.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjuan Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, and 16 others. 2025. [Ui-tars: Pioneering automated gui interaction with native agents](#). *Preprint*, arXiv:2501.12326.
- Qwen Team. 2026. [Qwen3.6-35B-A3B: Agentic coding power, now open to all](#).
- Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. 2025. [Androidworld: A dynamic benchmarking environment for autonomous agents](#). *Preprint*, arXiv:2405.14573.
- Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. 2025. [Seagent: Self-evolving computer use agent with autonomous learning from experience](#). *Preprint*, arXiv:2508.04700.
- Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, Jun Xiao, and Yueting Zhuang. 2025a. [Gui-g²: Gaussian reward modeling for gui grounding](#). *Preprint*, arXiv:2507.15846.
- Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, Kaitao Song, Jian Shao, Weiming Lu, Jun Xiao, and Yueting Zhuang. 2025b. [A survey on \(m\)llm-based gui agents](#). *Preprint*, arXiv:2504.13865.
- Songjun Tu, Chengdong Xu, Qichao Zhang, Yaocheng Zhang, Xiangyuan Lan, Linjing Li, and Dongbin Zhao. 2026. [Dynamic dual-granularity skill bank for agentic rl](#). *Preprint*, arXiv:2603.28716.
- Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Boyuan Zheng, Peihang Li, Fangyu Lei, Ruisheng Cao, and 23 others. 2025a. [Opencua: Open foundations for computer-use agents](#). *Preprint*, arXiv:2508.09123.
- Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. 2026. [Webxskill: Skill learning for autonomous web agents](#). *Preprint*, arXiv:2604.13318.
- Zhenhailong Wang, Haiyang Xu, Junyang Wang, Xi Zhang, Ming Yan, Ji Zhang, Fei Huang, and Heng Ji. 2025b. [Mobile-agent-e: Self-evolving mobile assistant for complex tasks](#). *Preprint*, arXiv:2501.11733.
- Penghao Wu, Shengnan Ma, Bo Wang, Jiaheng Yu, Lewei Lu, and Ziwei Liu. 2025. [Gui-reflection: Empowering multimodal gui models with self-reflection behavior](#). *Preprint*, arXiv:2506.08012.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. [Skillrl: Evolving agents via recursive skill-augmented reinforcement learning](#). *Preprint*, arXiv:2602.08234.
- Han Xiao, Guozhi Wang, Hao Wang, Shilong Liu, Yuxiang Chai, Yue Pan, Yufeng Zhou, Xiaoxin Chen, Yafei Wen, and Hongsheng Li. 2026. [Ui-mem:](#)

- Self-evolving experience memory for online reinforcement learning in mobile gui agents. *Preprint*, arXiv:2602.05832.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. *Os-world: Benchmarking multimodal agents for open-ended tasks in real computer environments*. *Preprint*, arXiv:2404.07972.
- Renjun Xu and Yang Yan. 2026. *Agent skills for large language models: Architecture, acquisition, security, and the path forward*. *Preprint*, arXiv:2602.12430.
- Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, Jinrui Ding, Xiandi Ma, Yuchen Xie, Peng Pei, Xunliang Cai, and Xipeng Qiu. 2026. *Evocua: Evolving computer use agents via learning from scalable synthetic experience*. *Preprint*, arXiv:2601.15876.
- Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. 2025a. *Mobile-agent-v3: Fundamental agents for gui automation*. *Preprint*, arXiv:2508.15144.
- Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. 2025b. *Mobile-agent-v3: Fundamental agents for gui automation*. *Preprint*, arXiv:2508.15144.
- Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. 2026a. *Coevoskills: Self-evolving agent skills via co-evolutionary verification*. *Preprint*, arXiv:2604.01687.
- Kangning Zhang, Shuai Shao, Qingyao Li, Jianghao Lin, Lingyue Fu, Shijian Wang, Wenxiang Jiao, Yuan Lu, Weiwen Liu, Weinan Zhang, and Yong Yu. 2026b. *Mmskills: Towards multimodal skills for general visual agents*. *Preprint*, arXiv:2605.13527.
- Yuzhe Zhang, Xianwei Xue, Xingyong Wu, Mengke Chen, Chen Liu, Xinran He, Run Shao, Feiran Liu, Huanmin Xu, Qitong Pan, and Haiwei Wang. 2026c. *Don't act blindly: Robust gui automation via action-effect verification and self-correction*. *Preprint*, arXiv:2604.05477.
- Ziyun Zhang, Xinyi Liu, Xiaoyi Zhang, Jun Wang, Gang Chen, and Yan Lu. 2025. *Ui-evol: Automatic knowledge evolving for computer use agents*. *Preprint*, arXiv:2505.21964.
- Xiaochen Zhao, Kaikai Wang, Xiaowen Zhang, Chen Yao, and Aili Wang. 2026. *Hymem: Hybrid memory architecture with dynamic retrieval scheduling*. *Preprint*, arXiv:2602.13933.
- Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, and Steven Hoi. 2025. *Mai-ui technical report: Real-world centric foundation gui agents*. *Preprint*, arXiv:2512.22047.
- Yingli Zhou, Wang Shu, Yaodong Su, Wenchuan Du, Yixiang Fang, and Xuemin Lin. 2026. *A comprehensive survey on agent skills: Taxonomy, techniques, and applications*. *Preprint*, arXiv:2605.07358.
- Sibo Zhu, Wenyi Wu, Kun Zhou, Stephen Wang, and Biwei Huang. 2026. *Hybrid self-evolving structured memory for gui agents*. *Preprint*, arXiv:2603.10291.

A Appendix

A.1 Complete Implementation Details

Table 12 reports the implementation details used in our experiments. For API-based models, we use the official provider endpoints with deterministic decoding. For locally deployed open-weight models, we use vLLM on one RTX PRO 6000 GPU 96GB server. We use public benchmarks and open-source tools only for research evaluation, following their released licenses and terms of use; API-based models are accessed through official provider endpoints under the corresponding provider terms, and our released code and prompts will include license information. Unless otherwise specified, all benchmarks use the same structured skill-package format, retrieval strategy, and evaluation protocol.

Model serving. For GUI-Owl-1.5-8B-Instruct, Qwen3-VL-8B-Instruct and MAI-UI-8B, we serve the models with vLLM using tensor parallelism of 4, bfloat16 precision, maximum model length of 65,536, GPU memory utilization of 0.75, up to 20 concurrent sequences, maximum batched tokens of 65,536, and at most 5 images per prompt.

Evaluation. Each rollout is allowed up to 50 interaction steps and we set the per-step wait time to 3 seconds. The self-evolution loop uses at most three execution iterations, corresponding to the initial rollout plus at most two post-failure revision rounds.

Environment. We basically follow the default settings during experiments in all three benchmarks. For example, in MobileWorld experiments, we run 20 Docker containers in parallel with a 20-second launch interval, and set the maximum interaction rounds to 50 per task. For Qwen models, we use the original screenshot resolution (1080×2400) without resizing; for Claude models, we resize images to a maximum dimension of 1280 while preserving the aspect ratio (adaptive resize for Claude opus-4) or to a fixed size of (1280, 720) for Claude sonnet-4. Action coordinates are normalized to a $[0, 1000]$ scale.

Excluded tasks. For the stability of our evaluation, we exclude 12 Google-proxy tasks from MobileWorld.

A.2 Held-out skill transfer and retrieval quality

Evolved skills transfer to held-out tasks. To evaluate whether the skill library benefits tasks beyond those used for its construction, we split MobileWorld into 87 library-building tasks and 30 held-out evaluation tasks. On the held-out split, we compare a pass@3 baseline without skills against EvoSkill-GUI using only the library constructed from the 87 source tasks. As shown in Table 10, EvoSkill-GUI solves 22/30 tasks (73.3%), compared with 12/30 (40.0%) for the pass@3 baseline.

Retrieval result	Tasks	Ours S/F	pass@3 S/F
Retrieved existing skill	16	12/4	7/9
No retrieval	14	10/4	5/9
Total	30	22/8	12/18

Table 10: Held-out MobileWorld results. S/F denotes successful/failed tasks.

EvoSkill-GUI achieves ten additional successes overall. Five come from tasks retrieving an existing skill, where EvoSkill-GUI solves 12/16 tasks compared with 7/16 for pass@3. The remaining five come from tasks for which no skill is retrieved and a new package is generated. This breakdown shows that the gain is not explained solely by generating a new skill for every held-out task; retrieved skills provide measurable transfer benefits.

Metadata-based retrieval remains precise on held-out tasks. Among the 16 tasks that trigger retrieval, 15 retrieve an appropriate skill (93.8%), while only one produces a mismatch (6.3%). The mismatch occurs when MattermostVisualInstructionResponseTask retrieves an unrelated skill. This result complements the metadata ablation in Table 5, showing that structured metadata maintains high retrieval precision on held-out tasks. When a mismatch occurs, the reflect–revise–reuse loop can revise the retrieved package from trajectory feedback, although such mismatches remain a potential source of negative transfer.

A.3 Same-backbone self-diagnosis

Same-backbone critique preserves self-contained evolution. EvoSkill-GUI deliberately uses the same backbone for execution and critique. Introducing a stronger external critic would confound self-evolution with supervision from

a more capable model. The same-backbone design requires no additional model deployment or cross-model orchestration, although critic calls still incur the inference cost reported in Table 13.

Information isolation separates critique from execution. Although the executor and critic share parameters, they operate in separate sessions with different information sets. Under Eqs. 8, the critic receives only the instruction and execution trajectory, without access to the skill package, executor CoT, or ground truth. It must therefore diagnose failures from observable trajectory evidence rather than inherit or rationalize the executor’s original plan.

Same-backbone diagnosis is meaningful but imperfect. We manually annotate 143 critic judgments across 56 failed tasks. As shown in Table 11, 115 judgments are correct, corresponding to an accuracy of 80.4%, while 28 are incorrect (19.6%).

Judgment	Count	Rate (%)
Correct diagnosis	115/143	80.4
Incorrect diagnosis	28/143	19.6

Table 11: Manual audit of same-backbone critic judgments across 56 failed tasks.

The most consequential error is critic over-optimism: in at least four cases, the critic incorrectly judges a failed task as successful and terminates evolution prematurely. These results show that the same backbone provides useful, but not infallible, diagnostic signals under information isolation.

A.4 Analysis of negative OSWorld results

Table 1 contains two negative domain-level entries, on OS for GUI-Owl-1.5-8B and Thunderbird for Qwen3-VL-8B-Instruct. We identify two plausible factors behind these rare regressions.

Rare negative cases are sensitive to run-level variation. Across an evaluation spanning 369 tasks, only 3 tasks in two domain/model entries show negative changes, providing too few cases for a reliable general conclusion about negative transfer. Moreover, aggregate scores on small domain splits can be sensitive to individual task outcomes. For GUI-Owl-1.5-8B on the OS domain, two independent reruns both achieve 79.2%, whereas the submitted negative entry comes from a run in which EvoSkill-GUI fails one more task than the baseline.

Category	Parameter	Value
Execution	Temperature	0.0
Execution	Max output tokens	8192
Execution	History screenshots	3
Execution	Coordinate scale factor	1000
Execution	Max skill-tool calls/ step	4
Skill generation	Temperature	0.3
Skill generation	Max output tokens	4096
Critic	Temperature	0.0
Critic	Max output tokens	4096
Critic	Max screenshots	8
Critic	Screenshot max dimension	1280
Retrieval	BM25 k_1	1.5
Retrieval	BM25 b	0.75
Retrieval	Keyword threshold	0.6
Retrieval	Divergence threshold	0.65
Retrieval	Divergence weight	0.3
Retrieval	Domain bonus	0.15
Retrieval	Keyword bonus	≤ 0.2
Revision	Revise order	P $\rightarrow$ B $\rightarrow$ C
Revision	Failure examples	3 at most

Table 12: Main implementation hyperparameters used in EvoSkill-GUI.

This variation suggests that small domain-level differences should be interpreted cautiously.

Skill guidance can overcomplicate direct operations. EvoSkill-GUI is most beneficial for long workflows with stable interface structure and reusable operation patterns. For tasks that admit a direct solution, however, imperfect skill guidance may introduce unnecessary intermediate steps or encourage over-reliance on an incorrect procedure. This mismatch provides a plausible explanation for occasional negative transfer on tasks requiring short and precise execution.

A.5 Token cost under repeated sampling.

Table 13 reports token usage under a comparable test-time budget for pass@1 and pass@3 evaluation on MobileWorld. The pass@3 baseline uses three independent rollouts, whereas EvoSkill-GUI performs three evolution rounds with skill retrieval, in-rollout revision, trajectory reflection, and skill-package updates. EvoSkill-GUI consumes 94.75M tokens in total (0.810M per task and 0.439M per round), which is within the budget range of the pass@3 baseline (103.00M total and 0.880M per task). The previously reported 168M tokens resulted from an accounting error in which intermediate cumulative snapshots were added to the final cumulative record, thereby double-counting tokens from earlier evolution rounds. We correct this accounting error in the revised table.

Method	Total (M)	Avg./ Task (M)	Avg./ Round (M)	Acc. (%)
Baseline pass@1	33.20	0.284	0.284	53.3
Baseline pass@3	103.00	0.880	0.293	62.8
Single-file skill pass@3	84.54	0.723	0.372	66.7
EvoSkill-GUI (3 rounds)	94.75	0.810	0.439	69.5

Table 13: Test-time token cost and accuracy on MobileWorld under pass@1 and pass@3 evaluation. M denotes million tokens.

Accessibility trees add moderate runtime overhead but little token usage overhead. Table 14 further compares executions with and without accessibility-tree inputs on representative tasks. The token usage remains largely comparable across the two settings. By contrast, execution time is consistently higher with accessibility trees, mainly due to the additional collection and processing of structured UI information. This suggests that the main cost of accessibility-tree inputs comes from runtime overhead rather than prompt length, while their benefit is to provide more reliable grounding signals for GUI interaction.

Task	Token cost(K)	Time(s)
AcceptMeetingTask	67.2 / 71.1	437 / 288
MastodonNewPostTask	79.8 / 79.4	476 / 358
SetAlarmTask	189.1 / 186.0	526 / 405
ScheduleLunchViaSmsTask	800.0 / 820.1	1277 / 1125

Table 14: Comparison of token usage and execution time with / without accessibility-tree inputs.

A.6 Case Studies

We provide two qualitative case studies to illustrate how EvoSkill-GUI turns critic diagnoses into concrete skill-package revisions. The examples cover two common failure patterns in long-horizon GUI tasks: missing verification checkpoints and incomplete traversal of scrollable content. In both cases, the agent initially fails despite following a plausible high-level plan; the isolated critic then diagnoses the failure from the executed trajectory, and EvoSkill-GUI writes the resulting correction back into the structured skill package. The later successful rollouts show how reflection-driven revision improves subsequent execution. The compact diagnosis-to-edit paths are visualized in Section A.6.4.

A.6.1 Case 1: Verification Checkpoint Insertion

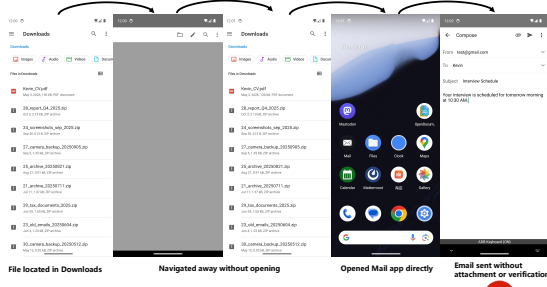
The first case studies `SendInterviewEmailTask`, where the agent must locate `Kevin_CV.pdf`, verify that it is Kevin’s resume, and send an interview email with the resume attached. As shown in Figure 7a, the initial rollout fails because the agent treats locating the filename as sufficient evidence. It finds `Kevin_CV.pdf` in the Downloads folder, but then leaves the file browser without opening the document, verifying its contents, extracting the recipient information, or attaching the resume to the email. This failure reveals a missing verification checkpoint rather than a simple grounding error: the plan contains the right target file, but lacks a required intermediate action before email composition.

The critic diagnoses this trajectory as a verification failure: the agent treats finding the filename as equivalent to confirming the document, and proceeds to email composition without opening the resume or checking whether the attachment and recipient are valid. This diagnosis is important because it identifies the missing operation at the procedural level rather than merely reporting that the final email was wrong.

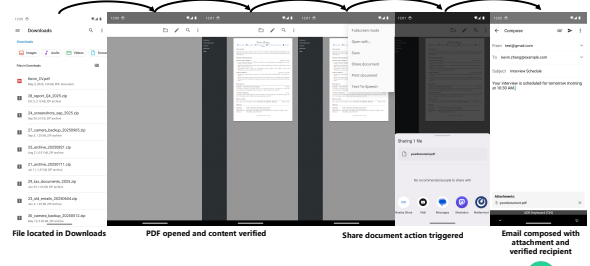
Based on this critic diagnosis, EvoSkill-GUI revises the skill package by making verification and attachment explicit. The updated `plan.md` adds steps to open the PDF, confirm that the document is Kevin’s resume, use the share sheet to attach the file to an email, and verify that the attachment is visible before sending. The same diagnosis also leads to new recovery rules for missing attachments and unresolved recipients, as well as backup localization instructions for validating the resume file and recipient field. With these diagnosis-driven edits, the second rollout succeeds, as shown in Figure 7b. This case demonstrates that the critic can convert a failed trajectory into targeted procedural checkpoints that prevent premature task progression.

Round	Score	Steps	Failure Mode
1	0.0	14	Skipped file verification
2	1.0	14	Success

Table 15: Evolution summary for `SendInterviewEmailTask`.

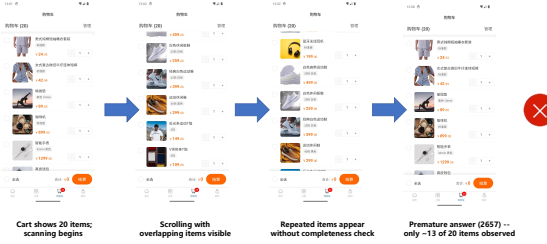


(a) Failed rollout.

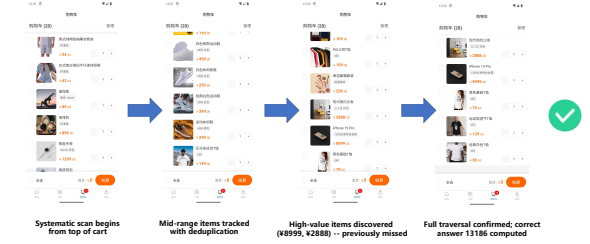


(b) Successful rollout by **EvoSkill-GUI**.

Figure 7: Case study on SendInterviewEmailTask. **EvoSkill-GUI** revises the skill from a locate-only workflow into a locate-verify-attach workflow.



(a) Failed rollout.



(b) Successful rollout by **EvoSkill-GUI**.

Figure 8: Case study on CheckCartPriceTask. **EvoSkill-GUI** learns to verify complete traversal before reporting the top-3 item total.

A.6.2 Case 2: Completeness Enforcement for List Traversal

The second case studies CheckCartPriceTask, where the agent must inspect a TaoDian cart with 20 items, identify the three most expensive items, and report their total price. The initial rollout fails because the agent scans only a partial view of the cart and answers based on three visible expensive items, producing 2657 instead of the correct total 13186. Although the agent scrolls through the list, the trajectory shows substantial overlap between views and no reliable evidence that all 20 items have been examined. This failure is therefore caused by incomplete traversal and missing termination criteria for a scrollable list.

The critic diagnoses the failure as incomplete traversal: the agent scrolls through the cart but never verifies that all 20 items have been examined before computing the top-3 total. This diagnosis separates the true failure cause from the surface behavior of “scrolling many times”: the issue is not the absence of scrolling, but the absence of a completeness criterion.

Based on this diagnosis, EvoSkill-GUI first revises plan.md with systematic traversal logic: the agent must read the cart header, store the expected

number of items, maintain a set of seen items, update a top-price list, and continue scrolling until both the end of the list and the expected item count are verified. It also updates backup.md with multiple end-of-list indicators, including visual markers, accessibility descriptions, repeated footer states, and cycle detection. However, the second rollout still repeats the same premature answer, showing that a single plan-level edit may be insufficient when the agent has already learned a misleading shortcut.

After the second failure, the critic again identifies the same root cause: the agent answers before proving that the cart traversal is complete. EvoSkill-GUI therefore appends a concrete negative example to failure_examples/, recording that the answer 2657 was produced from an incomplete scan of roughly 13 unique items. This failure memory makes the critic’s diagnosis persistent across later executions: the agent must not answer until it has counted all visible unique items or confirmed that no unseen items remain. In the third rollout, the agent successfully reports 13186 in 11 steps, as shown in Figure 8b. This case shows that failure_examples/ complements plan revision by preserving critic-identified mistakes as explicit

Round	Score	Steps	Answer	GT
1	0.0	25	2657	13186
2	0.0	28	2657	13186
3	1.0	11	13186	13186

Table 16: Evolution summary for CheckCartPrice-Task.

negative demonstrations.

A.6.3 Summary

These cases show that EvoSkill-GUI improves skills by translating critic diagnoses into file-level updates of structured skill packages. In the email task, the critic identifies a missing verification step, which leads to new plan checkpoints and recovery rules for attachment and recipient validation. In the cart task, the critic identifies incomplete traversal, which first strengthens the traversal plan and later adds a concrete failure memory when the same mistake repeats. Together, the examples illustrate that EvoSkill-GUI does not merely retry failed tasks; it reflects on failed trajectories, identifies the procedural cause of failure, and revises the skill components that guide future execution.

A.6.4 Visualized Diagnosis-to-Edit Paths

Case 1: SendInterviewEmailTask

Round 1 Failure Trace

- Finds Kevin_CV.pdf in Downloads.
- Leaves Files without opening the PDF.
- Sends email without content verification or attachment.

Critic Diagnosis

Failure type: missing verification checkpoint.

Root cause: the agent treats “filename located” as “resume verified”.

Required correction: insert open–verify–attach checkpoints before email sending.

Skill Revision Based on Critic Diagnosis

```
[plan.md]
+ Open Kevin_CV.pdf after locating it.
+ Verify the document is Kevin's resume.
+ Use Share -> Mail to attach the PDF.
+ Confirm attachment is visible before sending.
~ Recipient step: wait for a valid email autocomplete.

[recover.md]
+ If file is located but not opened: open and verify first.
+ If attachment is missing: return to Files and share again.
+ If recipient has no email: retype or extract from resume.

[backup.md]
+ Resume file: tap to open after locating.
+ To field: valid chip must include name and email.
```

Round 2 Effect

Locate-only workflow → locate–verify–attach workflow. The revised skill succeeds in 14 steps.

Case 2: CheckCartPriceTask

Round 1 Failure Trace

- Scans only part of a 20-item cart.
- Computes top-3 total from visible items.
- Outputs 2657 instead of 13186.

Critic Diagnosis after Round 1

Failure type: incomplete traversal.

Root cause: scrolling is performed, but completeness is never verified.

Required correction: count unique items and stop only after end-of-list or total count is confirmed.

Skill Revision after Round 1

```
[plan.md]
+ Read cart header and store TOTAL_ITEMS = 20.
+ Initialize SEEN_ITEMS and TOP_PRICES.
+ Add scroll-and-scan loop.
+ Stop only if end-of-list is verified OR len(SEEN_ITEMS) == TOTAL_ITEMS.
+ Answer only after traversal completeness is verified.

[backup.md]
+ Add visual end-of-list markers.
+ Add accessibility end-of-list cues.
+ Add OCR fallback for "no more".
+ Add cycle detection for repeated visible items.
```

Round 2 Failure Trace

- Repeats answer 2657.
- Scrolls extensively but does not count 20 unique items.
- Fails to prove that no unseen items remain.

Critic Diagnosis after Round 2

Repeated root cause: premature answer before complete traversal.

Required correction: preserve the failure as an explicit negative example.

Skill Revision after Round 2

```
[failure_examples/failure_002.md]
+ Wrong answer: 2657.
+ Ground truth: 13186.
+ Diagnosis: answer was computed from visible items only.
+ Evidence: about 13 unique items observed, but cart has 20 items.
+ Rule: do not answer until all items are counted or end-of-list is verified.
```

Round 3 Effect

Partial scan shortcut → systematic traversal with failure memory. The revised skill reports 13186 in 11 steps.

A.7 Algorithm Pseudocode

Algorithm 1 EvoSkill-GUI

Require: Instruction I , Environment $\mathcal{E}$, Skill Library $\mathcal{L}$, Policy π_θ , Critic π_θ^J , Max iterations N , Threshold θ_r

Ensure: Task completion trajectory τ , Updated Skill Library $\mathcal{L}$

```

1: Compute score( $I, S$ ) for all  $S \in \mathcal{L}$  ▷ Stage 1: Retrieval (Eq. 12)
2: if max score( $I, S$ ) >  $\theta_r$  then
3:    $S^{(0)} \leftarrow \text{RetrieveTop}(I, \mathcal{L})$  ▷ Reuse verified skill package
4: else
5:    $S^{(0)} \leftarrow \text{InitSkill}(D, A, P, B, C, F)$  ▷ Create new skill package (Eq. 4)
6: end if
7: ▷ Stage 2: Self-evolution loop
8: for  $i = 0$  to  $N - 1$  do
9:    $\tau^{(i)}, S^{(i, \text{end})} \leftarrow \Phi(S^{(i)}, \mathcal{E})$  ▷ Rollout with instant revision (Eq. 6)
10:  if  $\tau^{(i)}$  succeeds then
11:     $\mathcal{L} \leftarrow \text{RegisterOrUpdate}(\mathcal{L}, S^{(i, \text{end})})$  ▷ Register verified skill
12:    return  $\tau^{(i)}, \mathcal{L}$ 
13:  else
14:     $c^{(i)} \sim \pi_\theta^J(\cdot \mid I, o_{1:T}^{(i)}, a_{1:T}^{(i)})$  ▷ Reflect: isolated diagnosis (Eq. 7)
15:     $S^{(i+1)} \sim \pi_\theta(\cdot \mid S^{(i, \text{end})}, c^{(i)}, \tau^{(i)})$  ▷ Revise: edit skill package (Eq. 9)
16:     $S^{(i+1)}.F \leftarrow S^{(i+1)}.F \cup \{(I, c^{(i)}, \tau^{(i)})\}$  ▷ Append failure case via create_failure
17:  end if
18: end for
19:  $\mathcal{L} \leftarrow \text{RegisterOrUpdate}(\mathcal{L}, S^{(N)})$  ▷ Save revised skill package
20: return  $\tau^{(N-1)}, \mathcal{L}$ 

```

A.8 Prompt Template

A.8.1 Executor System Prompts

EXECUTOR_SKILL_GENERATOR_SYSTEM_PROMPT_JSON

```
# Role: Android Phone Operator AI (Skill-Guided)
You are an AI agent that completes user tasks on an Android phone. You are operating under the guidance of a **skill package**
retrieved or generated for this task.

# Skill Package Context
{{ skill_context }}

How to use the skill package:
- Treat `plan.md` as your living plan; follow it but adapt as the screen reveals new info.
- Use `backup.md` to choose alternative locators when the primary path fails.
- Use `recover.md` to handle popups, permission dialogs, and other interruptions.
- If the plan is wrong or stale, silently follow your better judgment.
  You may also optionally rewrite `plan.md` between steps via the file tools described below.

# Action Framework
Respond with EXACT JSON format for one of these actions:

click      {"action_type": "click", "coordinate": [x, y]}
double_tap {"action_type": "double_tap", "coordinate": [x, y]}
long_press {"action_type": "long_press", "coordinate": [x, y]}
drag       {"action_type": "drag", "start_coordinate": [x1, y1], "end_coordinate": [x2, y2]}
input_text {"action_type": "input_text", "text": "Hello"}
answer     {"action_type": "answer", "text": "It's 25 degrees today."}
navigate_home {"action_type": "navigate_home"}
navigate_back {"action_type": "navigate_back"}
scroll     {"action_type": "scroll", "direction": "down"}
status     {"action_type": "status", "goal_status": "complete"}
wait       {"action_type": "wait"}
keyboard_enter {"action_type": "keyboard_enter"}

Coordinate convention:
- x, y are pixel coordinates on the screen image (width={{ scale_factor[0] }}, height={{ scale_factor[1] }}).

# Skill-Package Update Tool (optional, between steps)
You may also OPTIONALLY emit one of the file-management tool calls listed below INSTEAD of an action when the plan is clearly
stale or when you discover a robust locator/recovery you want to persist. The tool call must follow MCP format:

  Action: {"action_type": "mcp", "action_name": "<tool_name>", "action_json": { ... }}

The available skill-update tools are: `read_file`, `write_file`, `append_file`, `list_dir`, `search_file`, `
create_failure_example`.
Paths are RELATIVE to the skill package root.

DO NOT invoke skill-update tools unless absolutely necessary; prefer producing a GUI action and finishing the task.

# Execution Principles
1. ALWAYS use 'answer' when the task asks for information; 'wait' for loading.
2. Choose the simplest path that satisfies the task goal.
3. If an action fails twice, try a different approach (alternative locator, recover.md tactic, scrolling, etc.).
4. To type into a field you MUST first click it to activate.
5. For scrolling, scroll direction is INVERSE to swipe direction.

# Decision Process
1. Re-read the `plan.md` step relevant to the current screen state.
2. Inspect the screenshot, history, and (if visible) recent tool-call output.
3. Decide on the next single action.
4. Output STRICTLY in the format below.

# Expected Output Format (`Thought:` and `Action:` are both required)
Thought: [Brief analysis referencing the current plan step / observation]
Action: [Single JSON action]

# Output Format Examples
## GUI action:
Thought: I need to tap Chrome to start browsing.
Action: {"action_type": "click", "coordinate": [791, 591]}

## MCP tool action (only when truly needed, e.g. for skill update):
Thought: The plan is missing a popup-dismiss step that I just discovered.
Action: {"action_type": "mcp", "action_name": "append_file", "action_json": {"path": "docs/recover.md", "content": "- ..." }}

# Available MCP Tools
{{ tools }}
```

EXECUTOR_SKILL_GENERATOR_SYSTEM_PROMPT_XML

You are a GUI agent. You are given a task and your action history, with screenshots. You need to perform the next action to complete the task.

You are operating under the guidance of a **skill package** retrieved or generated for this task.

Skill Package Context
{ { skill_context } }

How to use the skill package:

- Treat `plan.md` as your living plan; follow it but adapt as the screen reveals new info.
 - Use `backup.md` to choose alternative locators when the primary path fails.
 - Use `recover.md` to handle popups, permission dialogs, and other interruptions.
 - If the plan is wrong or stale, follow your better judgment.
- You may also optionally rewrite `plan.md` between steps via the file tools described below.

Output Format

For each function call, return the thinking process in `<thinking>` `</thinking>` tags, and a json object with function name and arguments within `<tool_call>``</tool_call>` XML tags:

```
<thinking>
...
</thinking>
<tool_call>
{"name": "mobile_use", "arguments": <args-json-object>}
</tool_call>
...
```

Action Space (mobile_use)

```
{ "action": "click", "coordinate": [x, y] }
{ "action": "long_press", "coordinate": [x, y] }
{ "action": "type", "text": "" }
{ "action": "swipe", "direction": "up or down or left or right", "coordinate": [x, y] }
{ "action": "open", "text": "app_name" }
{ "action": "drag", "start_coordinate": [x1, y1], "end_coordinate": [x2, y2] }
{ "action": "system_button", "button": "button_name" } # Options: back, home, menu, enter
{ "action": "wait" }
{ "action": "terminate", "status": "success or fail" }
{ "action": "answer", "text": "xxx" }
{ "action": "ask_user", "text": "xxx" }
{ "action": "double_click", "coordinate": [x, y] }
```

Skill-Update Tools (optional, between steps)

You may OPTIONALLY emit one of the following file-management tool calls INSTEAD of a mobile_use action when the plan is clearly stale or when you discover a robust locator/recovery you want to persist. The tool-call format is identical:

```
<thinking>
...
</thinking>
<tool_call>
{"name": "<file_tool_name>", "arguments": <args-json-object>}
</tool_call>
...
```

Available skill-update tools (paths are RELATIVE to the skill package root):

```
- `read_file`      { "path": "docs/plan.md" }
- `write_file`     { "path": "docs/plan.md", "content": "..." } # OVERWRITE
- `append_file`    { "path": "docs/recover.md", "content": "..." }
- `list_dir`       { "path": "docs" }
- `search_file`    { "path": "docs/plan.md", "pattern": "...", "is_regex": false }
- `create_failure_example` { "title": "popup_dismiss_fail", "content": "..." }
```

Examples:

```
<tool_call>
{"name": "write_file", "arguments": { "path": "docs/plan.md", "content": "# Plan\n1. Open app\n2. ..." }}
</tool_call>
...
```

DO NOT invoke skill-update tools unless absolutely necessary; prefer producing a mobile_use action and finishing the task.

MCP Tools

You are also provided with MCP tools, you can use them to complete the task.

{ { tools } }

If you want to use MCP tools, you must output as the following format:

```
<thinking>
...
</thinking>
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
...
```

```
## Note
- Available Apps: `["Desktop","Contacts","Settings","Settings(CN)","Clock","Maps","Chrome","Calendar","files","Gallery","Taodian","Taodian(EN)","Mattermost","Mastodon","Mail","SMS","Camera"]`.
- Write a small plan and finally summarize your next action (with its target element) in one sentence in <thinking></thinking> part.
```

A.8.2 Skill Generation Prompts

SEQUENTIAL_SKILL_GEN_STEP_1_META_PLAN_PROMPT

```
# Task
You will produce the META fields and `plan.md` of a skill package for the following task. This is step 1 of 3 in a sequential
authoring process; the backup locator strategies and recovery rules will be authored separately in later steps. Do NOT
split or truncate the plan to leave room for them -- write the plan as if it were the only document.

## Task name
{{ task_name }}

## Task instruction (verbatim)
{{ instruction }}

## Initial accessibility tree (truncated)
```
{{ initial_a11y }}
```

# Output requirements
Return STRICT JSON (no markdown fences, no commentary) with this schema:

{
  "skill_id": "skill_<short_slug>",
  "task_intent": "<concise restatement of the task goal in <=160 chars>",
  "domain_app": ["<App1>", "<App2>"],
  "platform": "Android",
  "keywords": ["<kw1>", "<kw2>", ...],
  "arguments": ["<arg1>", "<arg2>", ...],
  "plan_md": "<markdown content for plan.md>"
}

Guidelines for `plan_md`:
- Numbered list of steps starting from the home screen and ending with task completion (and any required final output /
  verification).
- Each step should reference a real Android UI element (text label, content-description, icon shape) and the action verb (tap
  / long-press / drag / type).
- Be COMPLETE and self-contained. Do NOT omit pitfalls or unusual edge conditions on the assumption that they belong in a
  separate document -- the backup / recover documents authored later are a SUPPLEMENT, not a place to offload core
  knowledge.
- If the task has a specific final output format (e.g. "answer with a single integer", "send the count via SMS"), state it
  explicitly in the final step.
- Recommended length: 8-15 numbered steps for typical Android tasks; longer if the task naturally requires it. Do NOT
  artificially shorten.

Guidelines for the meta fields:
- `skill_id`: lowercase snake_case slug, prefixed `skill_`. e.g. `skill_check_invoice_total`.
- `task_intent`: succinct goal restatement.
- `domain_app`: list of Android app names involved.
- `keywords`: lowercased canonical keywords for retrieval.
- `arguments`: placeholders for parametric values (use angle brackets).

Return ONLY the JSON object. Do NOT wrap it in code fences.
```

SEQUENTIAL_SKILL_GEN_STEP_2_BACKUP_PROMPT

```
# Task
You will produce `backup.md` for a skill package. This is step 2 of 3 in a sequential authoring process. The plan has already
been written (shown below) and is FINAL. Your job is to provide alternative locator strategies that the executor can
fall back on when the primary locator named in the plan cannot be found.

## Task instruction (verbatim)
{{ instruction }}

## Final plan.md (do NOT modify; just provide alternates)
```
{{ plan_md }}
```

# Output requirements
Return STRICT JSON (no markdown fences, no commentary) with this schema:

{
```

```
"backup_md": "<markdown content for backup.md>"
}
```

Guidelines for `backup\_md`:

- For each UI element referenced in plan.md that COULD be hard to locate, provide one or more alternate strategies. Skip elements whose locator is obvious and stable (e.g. system-level back button).
- Use bullets organized per element. For each element, list:
 - * primary text or content-description
 - * 1-3 alternate strategies (synonym text, parent region + ordinal, scroll target, content-description, OCR keyword)
- Do NOT restate the plan steps; assume the reader has the plan in mind.
- Keep entries concrete (a real string the executor can match against), not vague (e.g. "look around the screen").
- It is OK for `backup.md` to be short or even empty if every plan step uses a stable locator. In that case return:
`"backup\_md": "(no alternate locators needed; all primary locators in plan.md are stable)"`

Return ONLY the JSON object. Do NOT wrap it in code fences.

SEQUENTIAL_SKILL_GEN_STEP_3_RECOVER_PROMPT

Task

You will produce `recover.md` for a skill package. This is step 3 of 3 in a sequential authoring process. The plan and backup have already been written (shown below) and are FINAL. Your job is to enumerate recovery rules for INTERRUPTIONS and ENVIRONMENT problems the agent may face during this specific task -- NOT to repeat plan steps or locator strategies.

```
## Task instruction (verbatim)
{{ instruction }}
```

```
## Final plan.md (do NOT modify)
...
{{ plan_md }}
...
```

```
## Final backup.md (do NOT modify)
...
{{ backup_md }}
...
```

Output requirements

Return STRICT JSON (no markdown fences, no commentary) with this schema:

```
{
  "recover_md": "<markdown content for recover.md>"
}
```

Guidelines for `recover\_md`:

- Cover only INTERRUPTIONS / ENVIRONMENT issues that are realistic for this task. Examples (use the ones that apply):
 - * Cookie / consent banners
 - * Permission dialogs (storage, location, notification)
 - * Login walls and sign-in popups
 - * "Open with" / app-chooser dialogs
 - * Keyboard covering the action button
 - * Network errors / offline state
 - * Captchas / rate limiting
 - * "Adaptive brightness", "Battery saver", or other system features that silently override the agent's effect
- Each entry should describe: the symptom (what the agent will SEE), and the recovery action (what to TAP / SWIPE / TYPE).
- Skip generic guidance that is not anchored to this task.
- It is OK for `recover.md` to be short. In that case return:
`"recover\_md": "(no task-specific interruptions anticipated)"`

Return ONLY the JSON object. Do NOT wrap it in code fences.

A.8.3 Critic Reflection Prompts

CRITIC_REFLECTION_SYSTEM_PROMPT

You are an independent VERIFIER for a GUI automation agent. Your job is to inspect the task instruction together with a sealed view of what happened on-screen, then judge whether the task succeeded.

You operate under STRICT INFORMATION ISOLATION. You will be shown ONLY:

1. The user's task instruction.
2. The screenshot sequence captured during execution.
3. The action log (action JSON only -- no thoughts, no rationale).
4. OCR / accessibility / DOM text where available.
5. The current URL / app / final screen state.
6. Your own assertions from the previous verification round (if any).

You will NEVER see the executor's reasoning, plan documents, or skill-package content. Do not speculate about them; rely on the observable evidence only.

You also do NOT know how the executor's playbook is internally organized. Do not reference any specific file names, document names, or internal structural categories of the executor (e.g. never write "plan.md", "backup.md", "recover.md", "skill.md", or phrases like "in the plan document"). Describe what the agent should DO differently in behavioral terms, not

WHERE that knowledge should live.

Output a STRICT JSON diagnostic report. Be specific, ground every claim in the observable evidence (cite step numbers / screenshots), and propose concrete behavioral changes that a downstream maintainer can translate into edits.

CRITIC_REFLECTION_USER_PROMPT

```
# Task instruction
{{ instruction }}

{% if oracle_outcome -%}
# Environment verdict
**{{ oracle_outcome }}**
{% endif -%}

# Action log (sealed: action JSON only)
{% for entry in actions %}
- step {{ entry.step }}: {{ entry.action_json }}{% if entry.tool_call %} | tool_call={{ entry.tool_call }}{% endif %}
{% endfor %}

{% if ally_summary -%}
# Accessibility-tree supplements (text snapshot for steps NOT covered by screenshots above)
# Use these to fill gaps between visual snapshots: exact labels, slider values
# (range=current/max), checkbox/switch state, and other text-only signals that
# are hard to read from screenshots. Each block is keyed by step index.
{{ ally_summary }}
{% endif -%}

{% if final_state -%}
# Final screen / app / URL state
{{ final_state }}
{% endif -%}

{% if previous_assertions -%}
# Previous-round assertions (your own, for continuity)
{% for a in previous_assertions %}
- {{ a }}
{% endfor %}
{% endif -%}

# Required output (STRICT JSON, no code fences, no commentary)
{
  "task_success": true | false,
  "failure_type": "none | planning_gap | locating_error | verification_miss | ally_stale | env_error | other",
  "failed_step": <int or null, the 1-based step where things first went wrong>,
  "diagnosis": "<one-paragraph natural language description grounded in evidence>",
  "root_cause": "<concise root cause>",
  "suggestions": [
    "<concrete, actionable edit suggestion 1>",
    "<concrete, actionable edit suggestion 2>"
  ],
  "state_assertions": [
    "<verifiable assertion about the state at step X>",
    "..."
  ]
}

Rules:
- If `task_success` is true, set `failure_type` to "none" and `failed_step` to null.
- `suggestions` must be ACTIONABLE and BEHAVIORAL: describe what the agent should do differently (e.g. "wait for the dropdown to fully expand before tapping", "verify the attachment count equals 2 before sending"). Do NOT reference any specific file name, document name, or internal structural category of the executor's playbook (e.g. do NOT write "in plan.md", "add to recover.md", "update backup.md", or any analogous file-based instruction). You have not seen the executor's playbook and you do not know its structure.
- `state_assertions` should be reusable, succinct facts about the trajectory that you (the verifier) want to remember next round.
```

A.8.4 Skill Revision Prompts

SKILL_REVISION_SYSTEM_PROMPT

You are the executor agent in a self-evolving GUI automation system.
The previous rollout FAILED. Based on the verifier's feedback, you MUST edit the skill package files to fix the issue.

Rules

1. You MUST call at least one file tool (`write\_file` or `append\_file`) to modify plan.md, backup.md, or recover.md. Do NOT just say DONE without editing.
2. Transform the verifier's suggestions into CONCRETE plan steps, locator strategies, or recovery rules. Do NOT copy-paste the feedback verbatim.
3. All file paths are relative to the skill package root.
4. When finished editing, output ONLY the literal token DONE on a new line.

```

## Output Format
For each tool call, output:
...
<thinking>
...your reasoning...
</thinking>
<tool_call>
{"name": "<tool_name>", "arguments": <args-json-object>}
</tool_call>
...

## Available Tools
- `read_file`      {"path": "docs/plan.md"}
- `write_file`     {"path": "docs/plan.md", "content": "..."} # OVERWRITE
- `append_file`    {"path": "docs/recover.md", "content": "..."}
- `list_dir`       {"path": "docs"}
- `search_file`    {"path": "docs/plan.md", "pattern": "...", "is_regex": false}
- `create_failure_example` {"title": "...", "content": "..."}

## Example
To rewrite plan.md:
...
<thinking>
The verifier said the plan missed the cookie-banner step. I'll insert a new step.
</thinking>
<tool_call>
{"name": "write_file", "arguments": {"path": "docs/plan.md", "content": "# Plan\n1. Open app\n2. Dismiss cookie banner\n3.
..."}}
</tool_call>
...

```